Lab 11 · Generic Classes & Template Functions

LAB 11

Generic Classes & Template Functions

Learning Objectives

By the end of this lab, you should be able to:

- Explain what a template is and why it matters.
- Recognize the problem with hard-coded types (e.g., int) when the data could really be float, double, or char.
- Write a generic class using the template <typename T> syntax.
- Write a template function that works for any type.
- Instantiate a templated class with different types and observe how the compiler generates a separate version for each.

1. Why Do We Need Templates?

So far in this course, every variable in every class has had a FIXED type. If we wrote int a, b; in a Number class, that class could only ever store ints. If a user came along with floats, characters, or really large numbers, we would have to either copy the entire class and change the types — or live with broken behavior.

Templates solve this. A template lets you write a class (or a function) ONCE, leaving the type as a placeholder. The compiler then generates a separate version for every type you actually use. One source, many compiled versions, zero copy-paste.

> 📘 **Generic programming in one sentence**
> Write the algorithm or the data structure ONCE, and let the compiler stamp out a tailored version for each type you need. The logic stays the same; only the type changes.

2. Code 1 — The Problem with Fixed Types

Goal: write a Number class with two int data members and try to use it with int, float, large numbers, and char. Then watch the output go wrong.

The Code

```
#include <iostream>
```

```
using namespace std;

class Number {
    int a, b;
public:
    Number(int x, int y) { a = x; b = y; }

    int Max() { return (a >= b) ? a : b; }
    int Min() { return (a >= b) ? b : a; }
    int Add() { return a + b; }
};

int main() {
    Number n1(6, 7);
    cout << n1.Max() << " " << n1.Min() << endl;

    Number n2(5.8, 5.7);                    // floats → silently chopped to ints
    cout << n2.Max() << " " << n2.Min() << endl;

    Number n3(10000000000, 20000000000);    // too big for int → overflow
    cout << n3.Max() << " " << n3.Min() << endl;

    Number n4('r', 'c');                    // chars → stored as their ASCII ints
    cout << n4.Max() << " " << n4.Min() << endl;

    cout << n1.Add() << endl;
    cout << n2.Add() << endl;
}
```

What Goes Wrong

- *n1(6, 7)* — works perfectly. ints in, ints out.
- *n2(5.8, 5.7)* — the floats get **silently truncated** to 5 and 5. You wanted decimal numbers, you got integers.
- *n3(10000000000, 20000000000)* — these numbers are too large for a 32-bit int. They **overflow** and you get garbage.
- *n4('r', 'c')* — characters get stored as their ASCII integer codes (114 and 99). The output prints numbers, not letters.

Sample Output (with the bugs)

> ⚠ **Output — with wrong/truncated values**

```
7 6
5 5
1410065408 -2147483648
114 99
13
10
```

> ⚠ **The class isn't broken — the design is too narrow**
>
> Each issue above comes from one underlying problem: the class can ONLY hold ints. We need a class that can hold whatever type the user passes in — int, float, double, char, long long, even a string. That's exactly what templates give us.

3. Code 2 — Fixing It with a Generic Class

Goal: rewrite the Number class as a template so that a, b, and the return types all become whatever type the user wants.

The Code

```
#include <iostream>
using namespace std;

template <typename T>            // T is a placeholder for ANY type
class Number {
    T a, b;
public:
    Number(T x, T y) { a = x; b = y; }

    T Max() { return (a >= b) ? a : b; }
    T Min() { return (a >= b) ? b : a; }
    T Add() { return a + b; }
};

int main() {
    Number<int> n1(6, 7);
    cout << n1.Max() << " " << n1.Min() << endl;

    Number<float> n2(5.8, 5.7);
    cout << n2.Max() << " " << n2.Min() << endl;

    Number<long long> n3(10000000000LL, 20000000000LL);
    cout << n3.Max() << " " << n3.Min() << endl;

    Number<char> n4('r', 'c');
    cout << n4.Max() << " " << n4.Min() << endl;

    cout << n1.Add() << endl;
    cout << n2.Add() << endl;
}
```

Code Breakdown

Code	What it does
`template <typename T>`	← Says: "What follows is a TEMPLATE — T is a placeholder that the user will fill in." You can name it anything; T is the convention.
`T a, b;`	← The data members are of type T — whatever type the user picks when they create the object.
`Number(T x, T y)`	← The constructor takes two T values. Same parameter type as the data.
`T Max() / T Min() / T Add()`	← Each function returns a T. No more silent conversion to int.
`Number<int> n1(6, 7);`	← Instantiates the template with T = int. Compiler generates an int version of the class.
`Number<float> n2(5.8, 5.7);`	← Instantiates with T = float. Separate, independent float version is generated.
`Number<long long> n3(...);`	← long long can hold huge values — no overflow. The LL suffix marks the literals as long long.
`Number<char> n4('r', 'c');`	← T = char. Now Max returns a character, not an integer.

Expected Output

▶ **Output**

```
7 6
5.8 5.7
20000000000 10000000000
r c
13
11.5
```

What the compiler does behind the scenes

When you write *Number<int> n1;* and *Number<float> n2;*, the compiler generates TWO completely separate versions of the class — one with all the Ts replaced by *int*, another with all Ts replaced by *float*. You wrote the class once, the compiler did the duplication for you.

4. Template Functions

The same template trick works for free-standing functions too. Instead of writing one swap function for ints, one for floats, one for strings, you write one template swap function and the compiler generates the right version on demand.

Syntax

```
template <typename T>
return_type function_name(T param1, T param2, ...) {
    // body using T just like a normal type
}
```

Example — A Generic swap Function

```
#include <iostream>
using namespace std;

template <typename T>
void mySwap(T &x, T &y) {        // pass by reference so the swap actually sticks
    T temp = x;
    x = y;
    y = temp;
}

int main() {
    int a = 10, b = 20;
    mySwap(a, b);                                  // T deduced as int
    cout << "After int swap:    " << a << " " << b << endl;

    double p = 1.5, q = 2.5;
    mySwap(p, q);                                  // T deduced as double
    cout << "After double swap: " << p << " " << q << endl;

    string s1 = "Hello", s2 = "World";
    mySwap(s1, s2);                                // T deduced as string
    cout << "After string swap: " << s1 << " " << s2 << endl;
}
```

Expected Output

▶ **Output**

```
After int swap:    20 10
After double swap: 2.5 1.5
After string swap: World Hello
```

📘 **Notice — no <type> after mySwap**

For template FUNCTIONS, the compiler usually figures out T on its own by looking at the arguments you passed. *mySwap(a, b)* where a and b are ints → T is int. You could write *mySwap<int>(a, b)* explicitly if you wanted to, but you usually don't need to. For template CLASSES (like *Number<int>*), the angle brackets are required because the compiler has no arguments to deduce from.

5. Quick Recap — Templates at a Glance

- *template <typename T>* marks the line as a template; T is the placeholder type.

- **For a generic CLASS:** specify the type in angle brackets when creating an object — *Number<float> n;*
- **For a template FUNCTION:** usually no need for angle brackets — the compiler deduces T from the arguments.
- **One source, many compiled versions.** Every distinct type you use creates a separate generated version behind the scenes.
- **You can name the placeholder anything,** but *T* (for "Type") is the convention. For multiple placeholders use *T1, T2* or *T, U*.

6. Practice Tasks

Task 1 — Generic Calculator Class

Create a generic class Calculator that works for any numeric type.

Requirements

- Make it a template class: *template <typename T> class Calculator { ... };*
- Private data members: two values x and y of type T.
- Constructor that initializes both.
- Public functions (each returns T): *add()*, *subtract()*, *multiply()*, *divide()*.
- In main, create one *Calculator<int>* and one *Calculator<double>* and print the result of all four operations on each.

> **Hint for Task 1**
> For *divide()*, remember to handle *y == 0* — print an error and return a sensible default (like 0). Watch how *Calculator<int>* and *Calculator<double>* produce different output for the same division: 7/2 is 3 for int but 3.5 for double.

Task 2 — Template Function: largestOfThree

Write a single template function that returns the largest of three values, then use it with multiple types.

Requirements

- Function signature: *template <typename T> T largestOfThree(T x, T y, T z);*
- The function should return the largest of the three values.
- In main, call it with three ints, three doubles, and three chars. Print each result.

> 📘 **What you should observe**
>
> Calling *largestOfThree(3, 7, 5)* gives 7. Calling *largestOfThree(1.2, 4.5, 2.8)* gives 4.5. Calling *largestOfThree('a', 'k', 'g')* gives 'k' (because of ASCII ordering). One function, three different behaviors — the compiler stamps out a tailored version for each call.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST